Data Analytics Project:
IGN_Game_Reviews_Analysis

Dataset Used: ign

This document outlines a structured, end-to-end data analysis project based on game reviews from the IGN dataset. The project is divided into five scenarios, increasing in difficulty from basic data handling to advanced analysis and visualization, ensuring comprehensive skill coverage.

Project Goal: To analyze game review scores, platform popularity, and genre distribution over time using the ign.csv dataset.

The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (np.diff).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts).

🎯 Project Coverage

Category	Skills Covered
Data Preprocessing	Data Loading, Missing Value Handling (Mean/Mode), Type Conversion, Column Removal
Pandas Operations	Grouping, Filtering, Aggregation, Feature Engineering (score_category, editors_choice conversion)
NumPy Calculations	Array conversion, Yearly score growth calculation (np.diff)
Visualization	Line Chart, Bar Chart, Pie Chart, Histogram, Stacked Bar Chart, Saving Plots

Scenario 1: Basic Data Loading & Cleaning (🟢 Easy)

Description:

We load the IGN reviews dataset using **Pandas** and look at the first and last few rows to understand the data. Then we check the size (rows and columns) of the dataset. We remove the extra column **"Unnamed: 0"** because it is not useful. After that, we check for missing values in **score, genre, and platform** and fill them using mean and mode. Finally, we make sure the data types are correct for analysis

Task	Description
1.	Load the ign.csv dataset using Pandas.
2.	Display the first 5 rows (head()), last 5 rows (tail()), and the shape of the dataset.
3.	Remove the unnecessary column: "Unnamed: 0".
4.	Check for missing values in score, genre, and platform.
5.	Handle missing values: Fill the numeric column score with the mean, and the categorical column genre with the mode.
6.	Ensure correct data types: Convert score → float and release_year, release_month, release_day → integer.

Output Screen:

- **head()** - means showing the **first few rows** of a dataset (by default, first 5 rows). It helps you quickly see how the data starts and understand its structure.

- **tail()** - means showing the **last few rows** of a dataset (by default, last 5 rows). It helps you check how the data ends and verify if everything looks correct.

```
First 5 rows
    Unnamed: 0 score_phrase  ... release_month release_day
0            0      Amazing  ...             9          12
1            1      Amazing  ...             9          12
2            2        Great  ...             9          12
3            3        Great  ...             9          11
4            4        Great  ...             9          11

[5 rows x 11 columns]
==================================================
last 5 rows
        Unnamed: 0 score_phrase  ... release_month release_day
18620       18620         Good  ...             6          29
18621       18621      Amazing  ...             6          29
18622       18622     Mediocre  ...             6          28
18623       18623  Masterpiece  ...             6          28
18624       18624  Masterpiece  ...             6          28

[5 rows x 11 columns]
```

Conclusion:

After cleaning, the dataset becomes more accurate and easy to use. Missing values are handled properly, so there are fewer errors. Unnecessary columns are removed, and data types are corrected. Now the dataset is ready for analysis or visualization.

Scenario 2: Line Graph (Score Trend) + Save (🟢 Easy)

Description:

We group the dataset by release year to see how scores change over time. Then we calculate the average score for each year using **Pandas**. The results are converted into **NumPy** arrays for plotting. We create a **Line graph** with **years** on the **X-axis** and **average scores** on the **Y-axis**. Finally, we add a title, labels, and save the graph as an image.

Task	Description
1.	Group data by release_year.
2.	Calculate the average score per year using Pandas.
3.	Convert the resulting average scores and years into NumPy arrays.
4.	Plot a line graph with release_year on the X-axis and average score on the Y-axis.
5.	Add the title: "Average Game Score Over Years" and proper axis labels.
6.	Save the graph: plt.savefig("avg_score_trend.png").

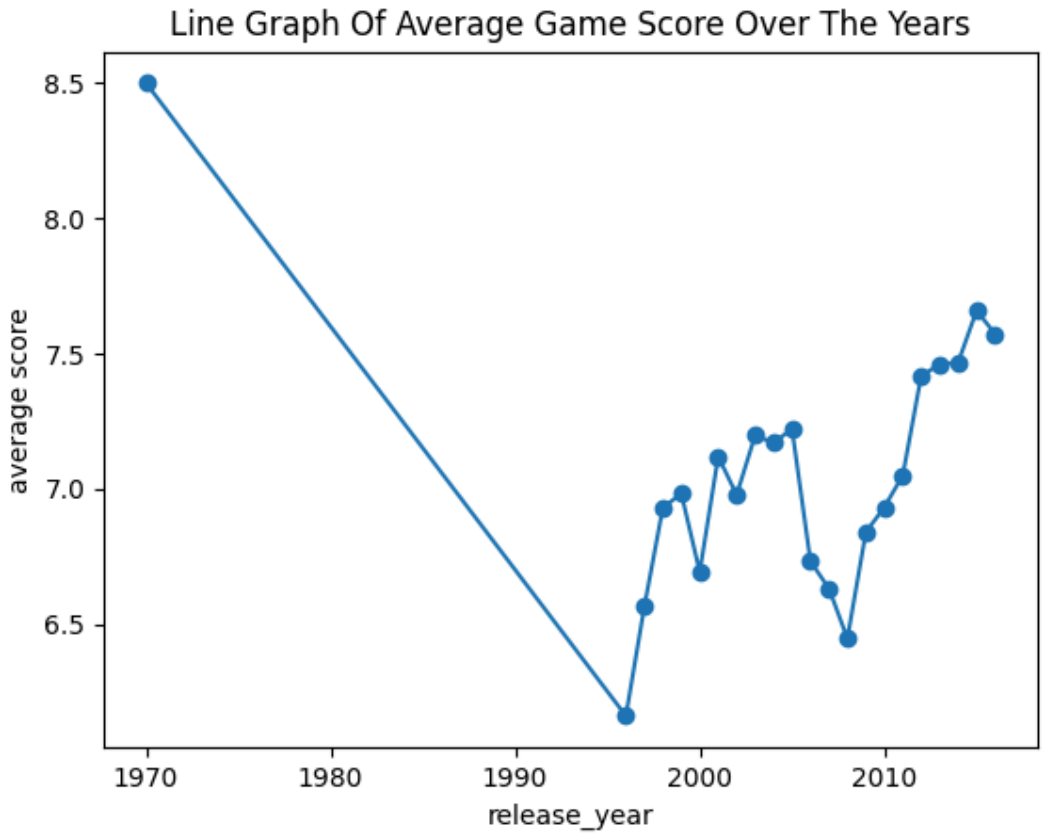
Output screen:

- Here, I am displaying yearly_average score by grouping all the scores.

```
(S2)yearly_avg
 release_year
1970    8.500000
1996    6.161745
1997    6.566341
1998    6.930678
1999    6.983484
2000    6.693589
2001    7.120494
2002    6.978526
2003    7.198246
2004    7.170712
2005    7.222426
2006    6.732864
2007    6.632547
2008    6.448146
2009    6.840368
2010    6.932869
2011    7.050326
2012    7.418327
2013    7.460489
2014    7.467598
2015    7.659452
2016    7.573451
Name: score, dtype: float64
```

Graph: Line Graph

- **Line Graph** - A **line graph** shows data points connected by lines. It is used to see **changes or trends over time**, like how game scores increase or decrease each year.



Conclusion:

The graph helps us clearly see how game scores change over the years. We can notice trends like increases or decreases in average ratings. This makes it easier to understand patterns in game quality over time. The saved graph can be used for reports or presentations.

Scenario 3: Filtering + Bar Chart + Save (🟡 Medium)

Description:

We first filter the dataset to keep only games with a **score greater than 7**. Then we count how many high-rated games each platform has. Using **Pandas**, we select the **top 10 platforms** with the most high scores. The data is converted into **NumPy arrays**

Task	Description
1.	Filter the dataset to include only games where score > 7.
2.	Count the number of high-rated games per platform.
3.	Select the top 10 platforms using Pandas.
4.	Convert the platform counts into NumPy arrays.
5.	Plot a bar chart with platform on the X-axis and count of games on the Y-axis.
6.	Rotate x-axis labels for readability.
7.	Save the graph: plt.savefig("top_platforms_bar.png").

Output Screen:

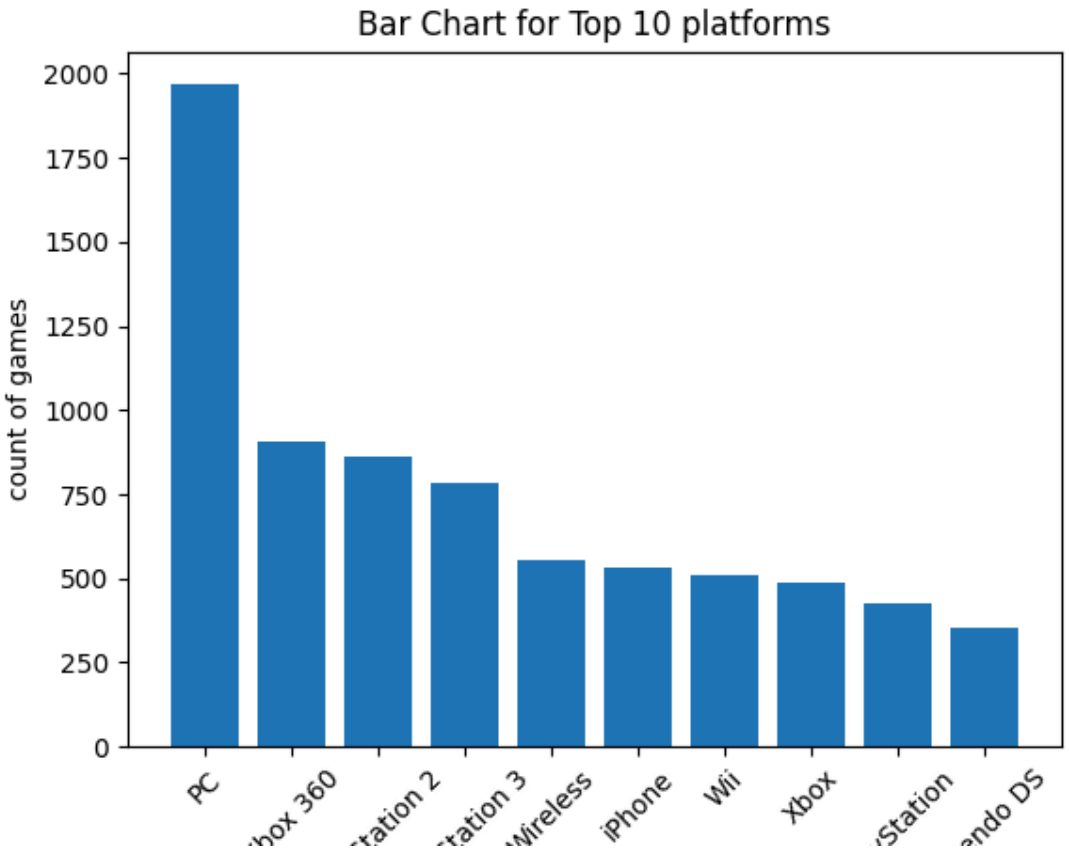
- Here, I am displaying dataset of score which is greater then 7 and counting the numbers of high-rated games per platform of top 10 platforms

```
==================================================
(S3) dataset preview
  score_phrase  ... release_day
0       Amazing  ...          12
1       Amazing  ...          12
2         Great  ...          12
3         Great  ...          11
4         Great  ...          11

[5 rows x 10 columns]
==================================================
(S3)high_rated
 platform
PC                1968
Xbox 360           909
PlayStation 2      864
PlayStation 3      783
Wireless           554
iPhone             531
Wii                512
Xbox               489
PlayStation        424
Nintendo DS        356
Name: count, dtype: int64
```

Graph: Bar Chart

- **Bar Chart** - A bar chart uses rectangular bars to represent values. It is used to compare different categories, like comparing the number of high-rated games on each platform.



Conclusion:

The chart clearly shows which platforms have the most high-rated games. This helps us understand which platforms perform better overall. It also makes comparison between platforms easy. The saved graph can be used for analysis or presentations

Scenario 4: Aggregation + Pie Chart + Save (🟡 Medium)

Description:

We count how many games are in each **genre** using the **dataset**. Then we select the **top 5 genres** with the highest number of games. We prepare **labels** (genre names) and **values** (counts) for visualization. A **pie chart** is created to show the distribution, with percentage labels added. Finally, we save the chart as an image.

Task	Description
1.	Count the total number of games per `genre`.
2.	Select the top 5 genres using Pandas.
3.	Prepare labels (genres) and values (counts) for the pie chart.
4.	Plot a pie chart using `genre` as labels and `count` as values.
5.	Add percentage labels using `autopct` (e.g., '%.1f%%').
6.	Save the graph: `plt.savefig("genre_distribution.png")`.

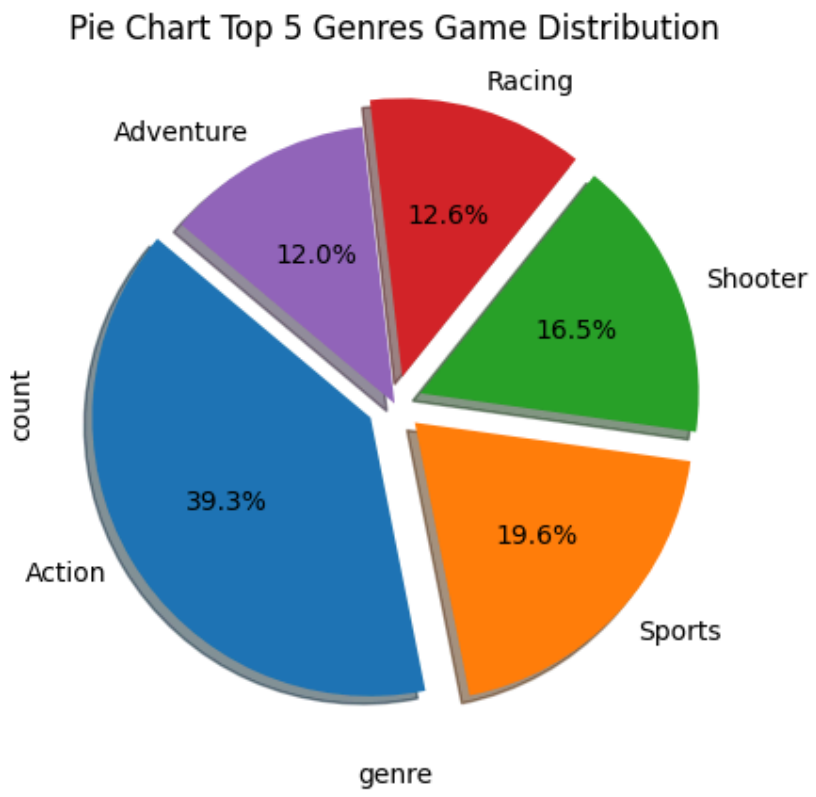
Output Screen:

- Here, I am counting the number of games per genre. And Selecting top 5 genres using Pandas.
-

```
(S4)genre_counts
 genre
Action                    3833
Sports                    1916
Shooter                   1610
Racing                    1228
Adventure                 1175
                          ...
Action, Strategy             1
Puzzle, Platformer           1
Music, Adventure             1
Sports, Golf                 1
Shooter, Adventure           1
Name: count, Length: 112, dtype: int64
==================================================
(S4)top 5 genres
 genre
Action        3833
Sports        1916
Shooter       1610
Racing        1228
Adventure     1175
Name: count, dtype: int64
```

Graph: Pie Chart

- **Pie Chart** - A pie chart is a round graph. It is divided into slices, and each slice shows a part of the total data.



Conclusion:

The **pie chart** helps us easily see which genres are most popular. It shows the proportion of each **genre** compared to others. This makes understanding the **dataset** simple and clear. The saved chart can be used for reports or presentations.

Scenario 5: Advanced Analysis + Multiple Graphs (🔴 Hard)

Description:

We create a new column **score_category** to group scores into Excellent, Good, and Average. Then we convert **editors_choice** into numbers for easy analysis. Using **NumPy**, we calculate how scores change each year. We plot different graphs like a **line graph, stacked bar chart,** and **histogram** to understand trends and distributions. Finally, we save all graphs as image files.

Task	Description
1.	Create a new column score_category: "Excellent" for score ≥ 9, "Good" for $7 \le$ score < 9, and "Average" for score < 7.
2.	Convert the editors_choice column: 'Y' $\rightarrow 1$, 'N' $\rightarrow 0$.
3.	Use NumPy's np.diff() to calculate the **yearly score growth** based on the average yearly scores from Scenario 2.
4.	Plot a **Line Graph** showing the trend of average score per release_year.
5.	Plot a **Stacked Bar Chart** showing the count of score_category per release_year.
6.	Plot a **Histogram** showing the distribution of score.
7.	Save the graphs: plt.savefig("score_trend.png"), plt.savefig("score_category_stacked.png"), plt.savefig("score_distribution.png").
8.	Identify: Which years had the highest scores, whether high scores increased over time, and if editors_choice correlates with high scores.

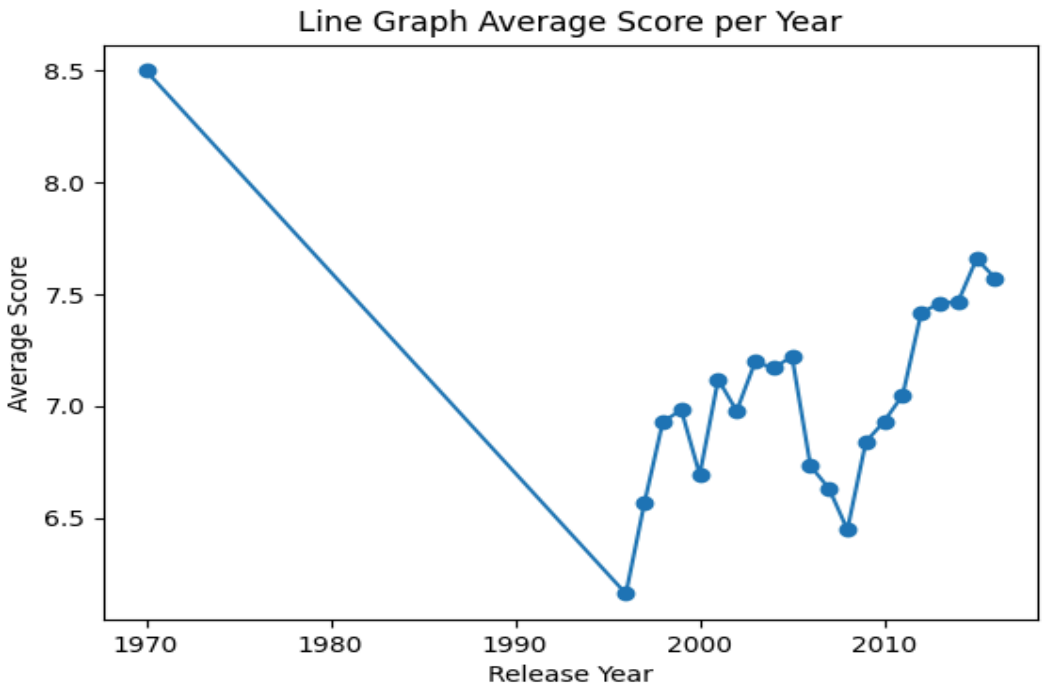
Output Screens:

- Here, I am displaying scores by categorizing the result as **Excellent, Good And Average.**

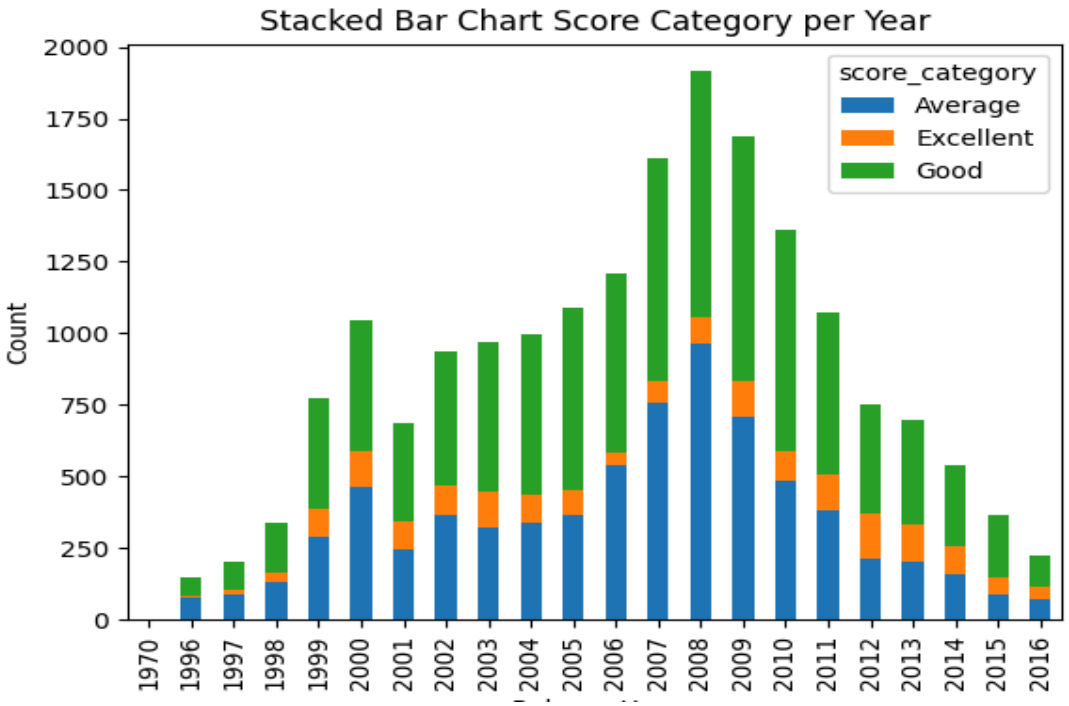
```
--------------------------------------------------
(S5)score_category
 0        Excellent
1        Excellent
2             Good
3             Good
4             Good
           ...
18620         Good
18621    Excellent
18622      Average
18623    Excellent
18624    Excellent
Name: score_category, Length: 18625, dtype: str
```

Graphs:

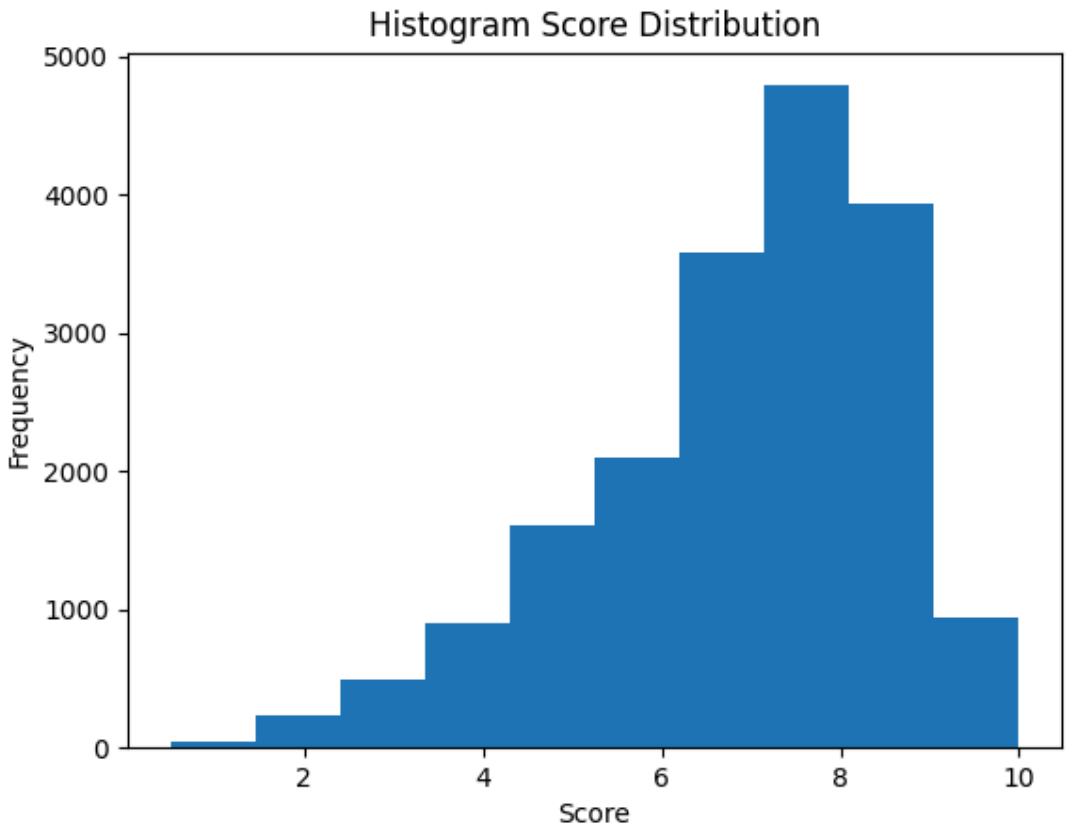
- **Line chart:** A graph that connects data points with a line to show trends or changes over time.



- **Stacked bar chart:** A bar chart where multiple categories are stacked on top of each other to show parts of a whole.



- **<u>Histogram</u>:** A chart that shows how data is distributed by grouping values into ranges (bins) and counting how often they occur.



<u>Conclusion</u>:

This analysis helps us understand game review patterns clearly. We can see score trends, category distribution, and overall score spread. It makes complex data easy to understand using graphs. The saved graphs can be used for reports or further study.

<u>Final Conclusion</u>:

This project helped analyze game review data from the IGN dataset using Python. We cleaned and prepared the data by handling missing values, fixing data types, and creating new useful features like score categories. We then explored trends using different visualizations such as line charts, bar charts, pie charts, and histograms. NumPy was used to study score growth over the years, which helped understand changes in game ratings over time.

What you have learned

I learned how to load and clean real-world data using Pandas. I understood how to handle missing values and convert data types correctly. I also learned how to use Matplotlib to create different types of graphs for data analysis. Finally, I learned how NumPy can be used to perform simple statistical calculations like yearly growth trends.